CAR PRICE PREDICTION

Project Report

CodeAlpha Internship — Task 3

A beginner machine learning project using Python to estimate the selling price of used cars.

1. About the Project

For this project, I worked with a used-car dataset and explored whether machine learning could help estimate how much a car might sell for. The dataset included details such as the car's year, current price, kilometres driven, fuel type and transmission. I used these details to train models to predict the selling price.

2. What I Wanted to Find Out

The main aim was to build a model that could make a reasonable estimate of a car's selling price. I also wanted to practise the main steps in a machine learning project: understanding the data, preparing it, training models and checking how well their predictions worked.

3. Understanding the Dataset

The original dataset had 301 rows and 9 columns. The value I wanted the model to predict was **Selling_Price**. The other columns described each car: **Car_Name**, **Year**, **Present_Price**, **Driven_kms**, **Fuel_Type**, **Selling_type**, **Transmission** and **Owner**. I found 2 duplicate rows and removed them, leaving 299 rows.

4. Preparing the Data

- I first checked the dataset to understand its columns and values.
- I checked for duplicate records and removed the 2 duplicates.
- I separated the information used to make predictions (X) from the value to predict (y, Selling_Price).
- Some columns contained text categories. I used one-hot encoding to convert these into numerical columns so the models could work with them.
- After encoding, the feature data contained 105 columns.
- I split the data into training and testing sets. 80% was used for training and 20% for testing, with random_state=42.

5. Models I Tried

I trained two models and compared their results. First, I used **Linear Regression** as a starting point. I then tried a **Random Forest Regressor**, which combines the results of many decision trees. For the Random Forest model, I used 200 trees, random_state=42 and min_samples_leaf=2.

6. Comparing the Results

I used three measurements to compare the models. R^2 gives an indication of how well the model explains differences in the selling prices. MAE shows the average size of the prediction error, and MSE gives more weight to larger errors. For MAE and MSE, lower values are better.

Model	R^2 score	MAE	MSE
Linear Regression	0.338751	2.622629	17.042543
Random Forest	0.579895	1.361225	10.827485

The Random Forest model gave better results than Linear Regression on my test data. It had a higher R^2 score and lower MAE and MSE. Its R^2 score was about 0.58, so there is still room to improve the model. It would not be accurate to say that it predicts every car price perfectly.

7. Which Features Mattered Most?

I also looked at the Random Forest feature-importance results to see which inputs the model relied on most. **Present_Price** had by far the highest importance, at about 0.9167. It was followed by **Year** at about 0.0569 and **Driven_kms** at about 0.0130. This tells me which features were most useful to this model; it does not prove that they directly cause a car's selling price to change.

Feature	Importance
Present_Price	0.916723
Year	0.056866
Driven_kms	0.012960
Transmission_Manual	0.002736
Car_Name_fortuner	0.002281
Car_Name_innova	0.002167

8. Testing a Prediction

I used the Random Forest model to make a prediction for one example from the test data. The model predicted a selling price of approximately **9.6355**, while the actual value was **8.99**. The difference was about **0.6455** in the dataset's original price units.

9. What I Learned

This project gave me practice with the full process of a basic machine learning task. I learned how to inspect a dataset, deal with duplicate rows, turn category values into numbers, split data into training and testing sets, train regression models and compare their performance. I also learned that trying more than one model is useful because different models can perform differently on the same data.

10. Conclusion

Overall, the Random Forest model performed better than Linear Regression for this dataset and test split. However, its results show that the prediction task could still be improved. In future, I could try tuning the model further, testing other algorithms, and using cross-validation to get a more reliable picture of how it performs across different data splits.

Tools Used

Python, pandas, NumPy, Matplotlib, Seaborn and scikit-learn.

Note: The figures in this report are based on the results recorded during the project. The dataset's currency or price unit has not been stated because it was not confirmed. Please check the report against the final notebook before submitting.